Sovereign AI Factory — Sovereignty Control Tower

Our first implementation target should be the French healthcare scenario, but the architecture should be reusable for other customers and sectors.

PART 1 — Overall technical solution

1. What exactly are we building?

Imagine a French healthcare organization wants to operate its entire IT infrastructure inside its own premises.

It wants:

- Its own physical servers.
- Its own GPU infrastructure.
- Its own AI models.
- Its own databases and storage.
- Its own identity and access management.
- Its own network and security.
- Its own backups and disaster recovery.
- Its own monitoring and governance.
- No mandatory public cloud dependency.

We provide the infrastructure and platform stack using Dell and NVIDIA hardware, along with Nutanix and OpenShift technologies.

Our Control Tower sits above this stack.

The complete sovereign platform

EY Sovereignty Control Tower

Governance · Assurance · Monitoring · AI Governance · Reporting

↓

Control Engine
44 baseline controls and 8 dimensions

Evidence Engine
Evidence collection and verification

Infrastructure Engine
Hardware and cluster telemetry

AI Governance
Model and RAG provenance

Identity Engine
Authentication and permissions

Workflow Engine
Exceptions and approvals

↓

Integration and Collector Layer

Redfish · Prism API · OpenShift API · NVIDIA DCGM · Prometheus · SIEM

↓

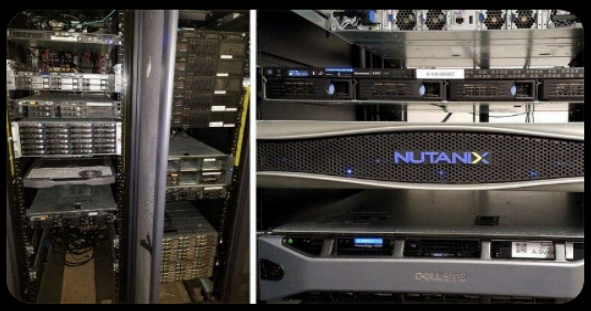
Sovereign Infrastructure



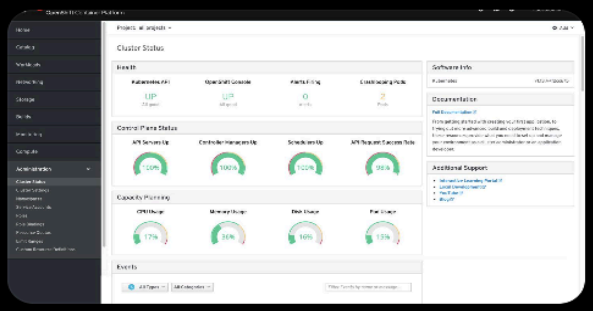
Dell PowerEdge
Compute and hardware



NVIDIA GPUs
AI acceleration



Nutanix
Virtualization and storage



OpenShift
Container platform

Physical infrastructure remains within the customer's controlled site.

2. The eight sovereignty dimensions

These must be retained as the foundation of the application.

Dimension	What the platform will assess
Governanec	Policies, accountability, jurisdiction and ownership
Data	Data residency, classification, encryption and access
Intelligence	Model ownership, AI governance and provenance
Compute	Server infrastructure, GPUs and workloads
Operations	Administration, monitoring and privileged access
Network	Isolation, segmentation and egress
Supply Chain	Software provenance, approved images and vendors
Continuity	Backups, disaster recovery and resilience

Every control, evidence record, exception, maturity assessment and report should link back to one of these dimensions.

PART 2 — How we will use Dell and NVIDIA hardware

This is the most important technical section to explain to your team.

3. Dell hardware — the infrastructure foundation

Based on the earlier design information, the expected hardware includes Dell XC760xa and R760 servers. We need the team to confirm the actual inventory before finalizing the deployment topology.

We should divide Dell integration into two separate responsibilities.

A. Physical hardware management

Use Dell iDRAC with Redfish API.

iDRAC is the out-of-band management controller built into PowerEdge servers. It lets our platform retrieve server information without needing to install an agent inside every operating system.

What we can collect:

Information	Purpose in Control Tower
Server model and service tag	Hardware inventory
CPU and memory	Compute capacity
Storage and RAID	Storage health

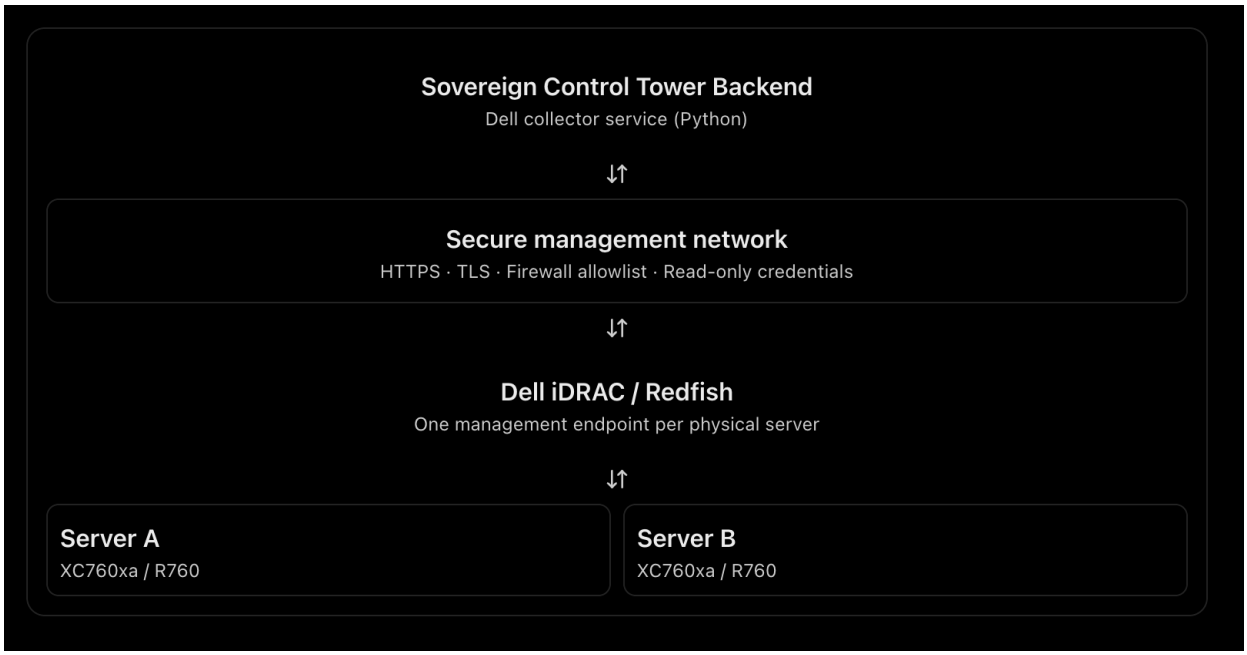
Temperature	Thermal health
Power consumption	Energy monitoring
Fans and power supplies	Hardware health
Firmware versions	Supply-chain and patch governance
Lifecycle logs	Hardware incidents
GPU-related telemetry	Hardware monitoring, where supported

Dell provides Redfish-based REST interfaces using JSON and OData.

Dell US+1

B. How our application connects to Dell

The connection should look like this:



Example endpoint

The standard Redfish service root is:

```
None
GET https://<IDRAC-IP>/redfish/v1/
```

Typical resource paths include:

```
None
GET /redfish/v1/Systems
GET /redfish/v1/Chassis
GET /redfish/v1/Managers
GET /redfish/v1/UpdateService/FirmwareInventory
```

Exact resource availability varies with iDRAC generation, firmware and hardware model. We should discover supported resources from the Redfish service root and its links rather than assume every endpoint exists.

Example Python connection

This is the type of collector we will build:

```python
import requests
from requests.auth import HTTPBasicAuth

IDRAC_URL = "https://<IDRAC-IP>"

response = requests.get(
    f"{IDRAC_URL}/redfish/v1/Systems",
    auth=HTTPBasicAuth(
        "<READ_ONLY_USERNAME>",
        "<PASSWORD>"
    ),
    verify="/etc/ssl/certs/idrac-ca.pem",
    timeout=15
)

response.raise_for_status()

systems = response.json()

print(systems)
```

For production, credentials must come from Vault, TLS certificates must be validated, and responses must be normalized and logged. Do not disable certificate verification.

What I need from the Dell team:

1. iDRAC IP addresses or DNS names.
2. iDRAC generation and firmware version.
3. Redfish API access enabled.

4. Read-only service account.
5. CA certificates or approved trust chain.
6. Management VLAN and firewall access.
7. Confirmation of licensing requirements for telemetry features.

We can use ordinary Redfish inventory and health resources as our starting point. Advanced streaming telemetry may require additional iDRAC licensing and configuration, depending on the server generation.

4. NVIDIA hardware — the AI acceleration foundation

Our expected GPU configuration includes NVIDIA L40S accelerators, but the final GPU model, number of GPUs, memory and server allocation must be confirmed.

We need to use NVIDIA in two ways:

1. To run AI workloads.
2. To monitor and govern the GPU infrastructure.

A. Running AI workloads

The proposed software stack:

Component	Responsibility
NVIDIA Drivers	GPU hardware communication
NVIDIA Container Toolkit	GPU access for containers
NVIDIA GPU Operator	GPU software lifecycle on Kubernetes

KServe	Model serving and inference deployment
vLLM	LLM inference engine
Prometheus	Metrics collection
NVIDIA DCGM	GPU health and telemetry
DCGM Exporter	Expose GPU metrics

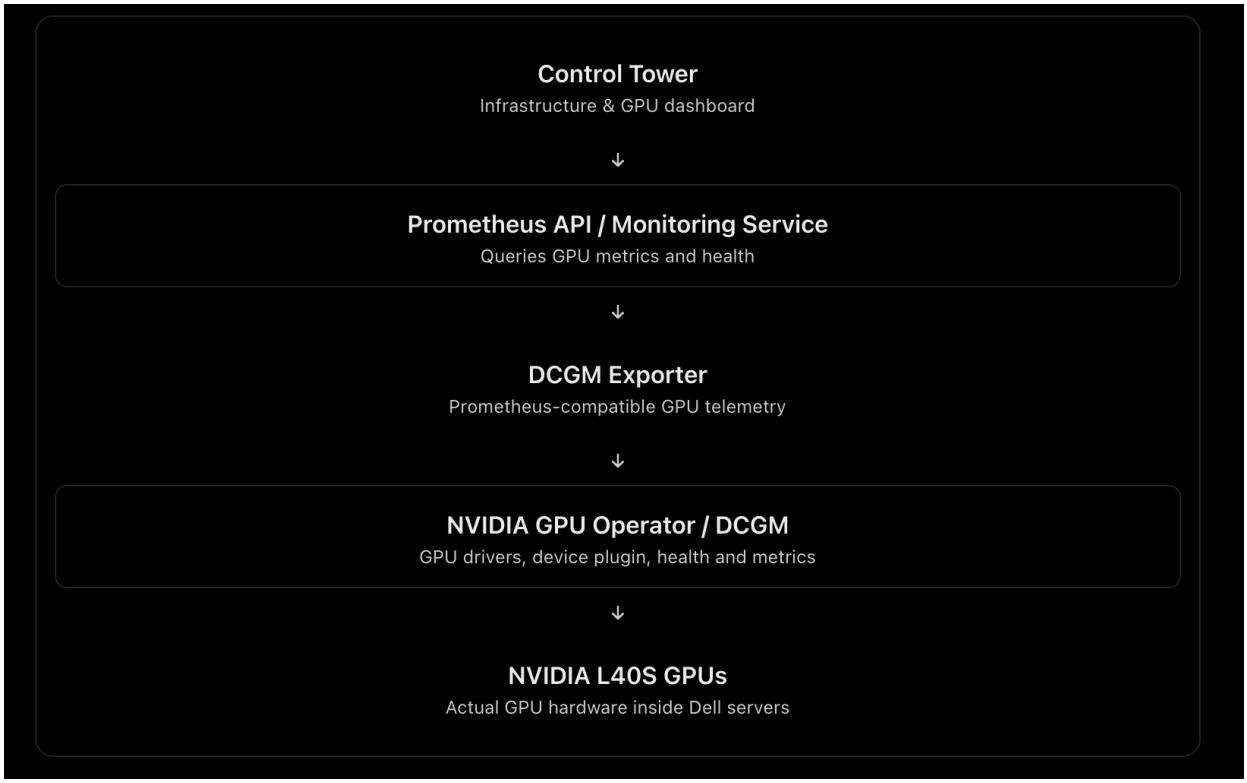
The NVIDIA GPU Operator can manage the required driver, device plugin, container toolkit and monitoring components on supported OpenShift environments.

NVIDIA Docs+1

B. How our application connects to NVIDIA

We should not connect the frontend directly to NVIDIA hardware.

Instead:



What GPU metrics will our dashboard display?

Metric	Example dashboard usage
GPU utilization	Workload utilization
GPU memory used	Capacity monitoring
GPU temperature	Thermal health
GPU power usage	Power consumption

GPU health	Operational state
GPU model	Inventory
Driver version	Software compliance
GPU-to-workload mapping	Resource attribution

Example Prometheus query:

```
None
DCGM_FI_DEV_GPU_UTIL
```

For memory:

```
None
DCGM_FI_DEV_FB_USED
```

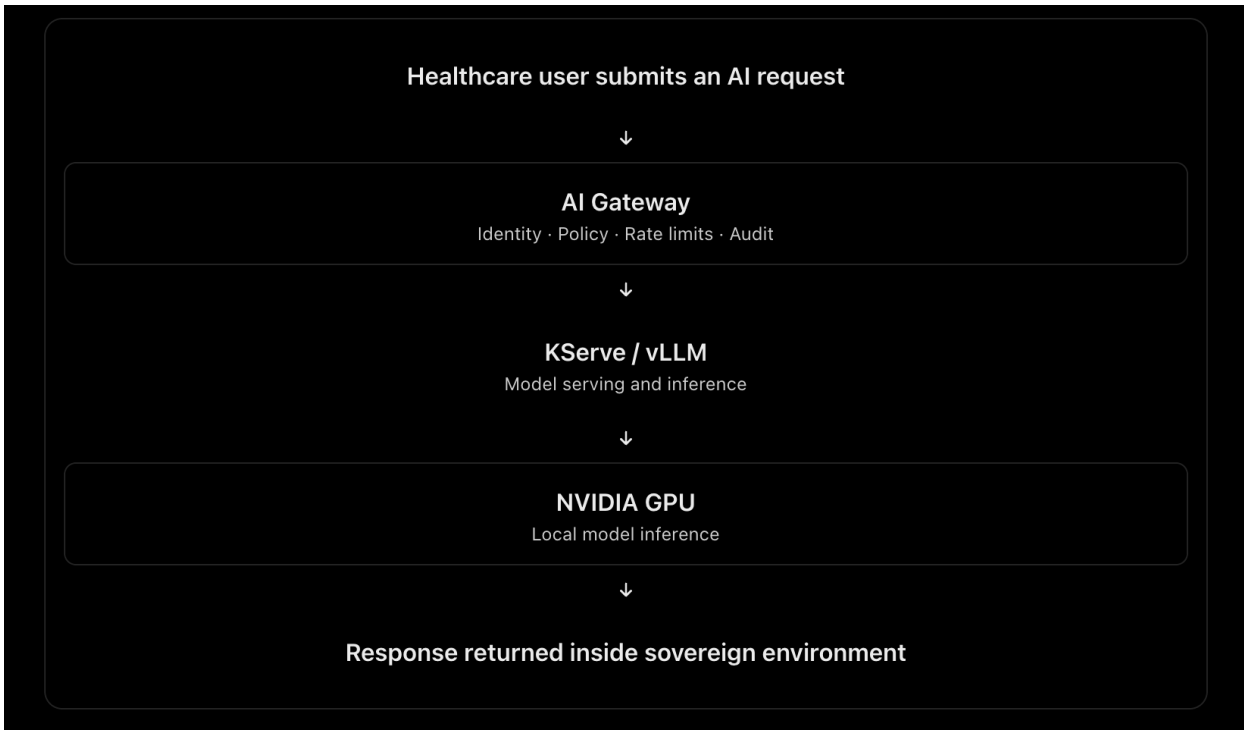
For GPU temperature:

```
None
DCGM_FI_DEV_GPU_TEMP
```

These are examples of available DCGM metrics; exact names and labels depend on the installed exporter and selected metric configuration.

C. What happens when a user runs an AI model?

This is another workflow we should include.



For healthcare, we must additionally ensure that the AI workflow is not automatically treated as clinically safe merely because it is sovereign. Clinical validation, human oversight, data protection and applicable medical-device requirements are separate concerns.

5. Nutanix — the virtualization and storage layer

Nutanix is important because it manages the infrastructure on which our platform and other customer workloads may run.

Our expected components:

- Nutanix AOS — distributed storage and infrastructure services.
- AHV — hypervisor.
- Prism Central — centralized management.
- Nutanix Flow — network security and segmentation, depending on deployment and licensing.

We should use Prism Central APIs for infrastructure inventory and operational state.

Our collector should retrieve:

- Cluster inventory.
- Hosts and hypervisor status.
- Virtual machines.
- Storage capacity and usage.
- Cluster alerts.
- Hardware and VM performance data.
- Network security policy information where exposed by the installed product and version.

The integration is:

```
Control Tower → Nutanix collector → Prism Central API → Nutanix cluster
```

Important architecture decision: Nutanix and OpenShift are not automatically one single management layer. The team must define whether OpenShift runs on AHV VMs, on bare-metal Dell nodes, or using another supported topology. That decision determines how we monitor the GPU nodes and which collector owns each resource.

For Prism Central, ask the Nutanix team to provide the installed release, supported REST API version, API documentation, endpoint, service account and RBAC scope.

6. OpenShift — the application and AI workload platform

OpenShift will be our primary container orchestration environment, assuming this is the selected platform in the final infrastructure design.

We will deploy the Control Tower itself on OpenShift, subject to the cluster topology and security approval.

It will host:

- Frontend application.
- Backend APIs.

- Collector workers.
- PostgreSQL.
- Evidence processing services.
- AI gateway.
- KServe and vLLM.
- Monitoring integrations.

We will connect using:

```
None
oc login https://<OPENSHIFT-API-ENDPOINT>:6443
```

Our backend will use the Kubernetes API and appropriately scoped service accounts.

For monitoring, we should reuse OpenShift's platform monitoring and user-workload monitoring where appropriate instead of creating duplicate monitoring systems. OpenShift documents a dedicated mechanism for monitoring user-defined projects.

Red Hat Documentation

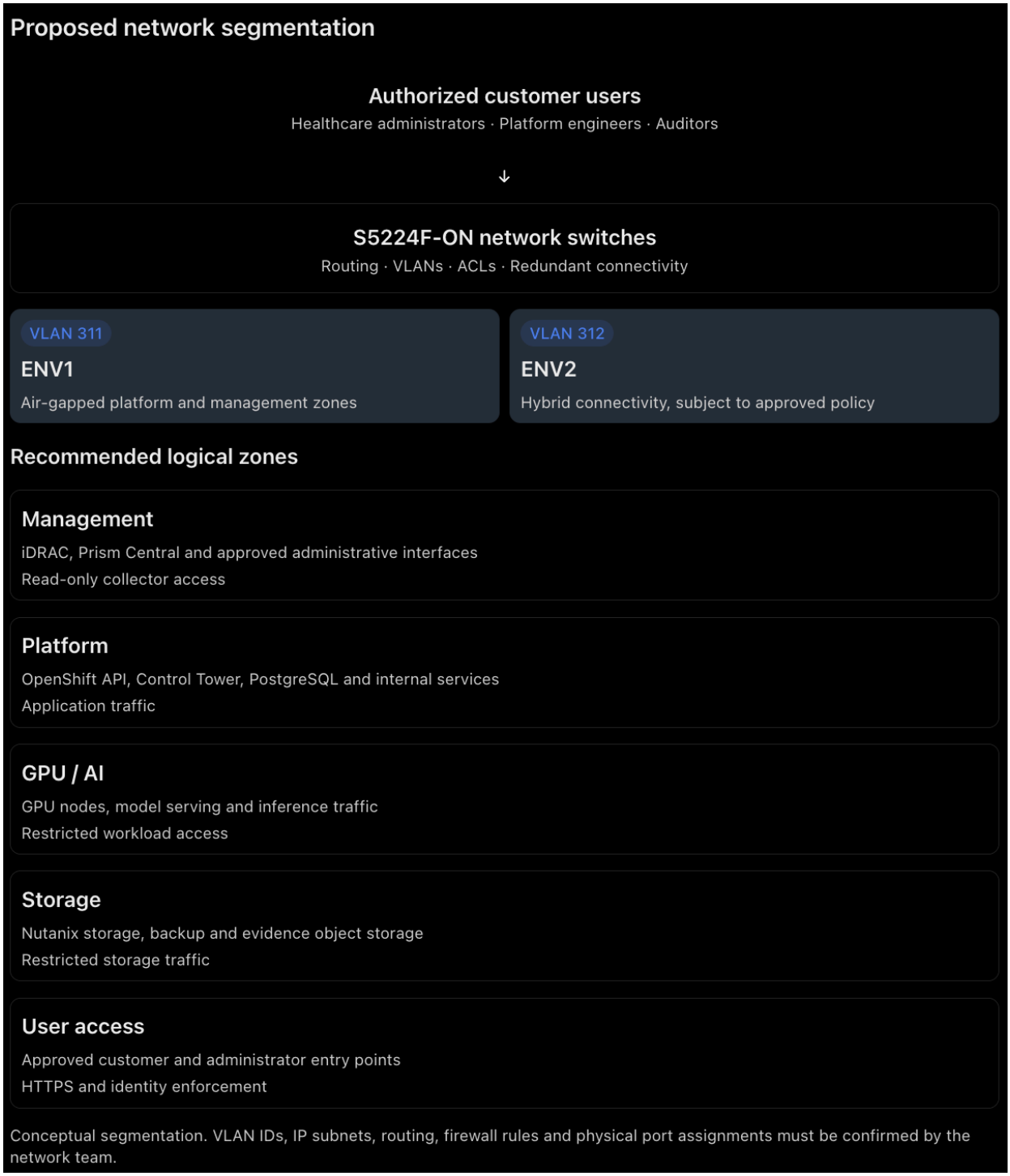
A critical point: We should not give our application cluster-admin privileges simply to read metrics. We will define individual service accounts and read-only RBAC roles for each collector.

PART 3 — How all hardware will be connected

7. The physical and logical network topology

This is the topology I recommend proposing to the infrastructure team.

We should separate the network into management, platform, storage, GPU/workload and user-access zones.



Proposed network requirements

Traffic	Suggested destination	Purpose
User → application	HTTPS 443	Dashboard access
Backend → iDRAC	HTTPS 443	Dell inventory
Backend → Prism Central	Approved HTTPS API port	Nutanix inventory
Backend → OpenShift API	TCP 6443	Kubernetes API
Prometheus → DCGM Exporter	TCP 9400	GPU telemetry
Backend → Prometheus	Approved HTTPS endpoint	Metrics queries
Services → PostgreSQL	TCP 5432	Application database
Services → Vault	Approved HTTPS endpoint	Secrets and signing

OpenShift → internal registry			Approved endpoint	HTTPS	Container images

These are proposed integration ports, not blanket firewall permissions. The network team must approve exact source/destination pairs and restrict traffic to the required interfaces.

Air-gapped environment requirements

For ENV1, we need to ensure:

- No runtime dependency on public CDN resources.
- No public container image pulls.
- No external telemetry or analytics.
- Internal DNS and NTP.
- Internal certificate authority or approved certificate chain.
- Internal package and container registries.
- Offline backups of installation artifacts.
- Signed offline software updates.
- A controlled process for importing patches and model artifacts.

We should also have a separate, controlled administrative path for accessing iDRAC. Do not expose management interfaces to ordinary application users.

8. The endpoint integration strategy

This is the exact integration contract I would ask your team to provide.

Source	Integration method	What we need from team	What we build
Dell iDRAC	Redfish REST API	IP, credentials, certificate	Dell collector

Nutanix Prism	REST API	Prism endpoint, version, credentials	Nutanix collector
OpenShift	Kubernetes REST API	API URL, CA, service account	OpenShift collector
NVIDIA GPU	DCGM Exporter / Prometheus	Metrics endpoint and access	GPU collector
Keycloak	OIDC and Admin API	Realm, client and permissions	Identity integration
Vault	HTTPS API	Vault address and auth method	Secrets integration
Quay	Registry API	Endpoint and read-only token	Image provenance collector
ACS	Product API	Endpoint and RBAC	Security collector
S5224F-ON	Supported switch management interface	Management IP, firmware, supported interface	Network collector
SIEM	Vendor-specific API or syslog	Product, endpoint, event format	Security event integration

Dell Redfish and NVIDIA DCGM have documented integration mechanisms. For Nutanix, the precise Prism API paths and versions must be selected against the team's installed release; I would not hardcode API assumptions before receiving its API specification.

One important design decision: Push versus Pull

We should support both collection approaches.

Pull-based collection

Our collector periodically requests information from the source.

Example:

```
None
Control Tower Collector

        |

        | HTTPS request

        v

Dell iDRAC / Prism / OpenShift

        |

        | JSON response

        v

Evidence Processing
```

Use this for inventory, configuration and periodic health checks.

Push-based collection

A source or exporter sends telemetry into an internal monitoring pipeline.

```
None
NVIDIA DCGM Exporter

        |

        v

Internal Prometheus

        |

        v

Control Tower Metrics Service
```

Use this for high-frequency telemetry.

For the initial version, I recommend:

- Dell: Pull.
- Nutanix: Pull.
- OpenShift: API-based collection and metrics.
- NVIDIA: DCGM Exporter with Prometheus scraping.
- Security events: Push or event streaming if supported.
- Evidence: Collector-produced records sent to our evidence ingestion API.

This reduces unnecessary direct connections and keeps the Control Tower's data collection responsibilities clear.

9. The most important part: How hardware telemetry becomes assurance

We must not simply display GPU utilization and call it compliance.

We need a proper processing pipeline.

Evidence processing lifecycle

1. Source collection
   Dell, Nutanix, OpenShift, NVIDIA and security sources provide configuration, inventory or telemetry.
2. Normalization
   Convert different source formats into one standard evidence structure.
3. Validation
   Validate schema, source identity, timestamps and required fields.
4. Integrity
   Calculate SHA-256, sign records and retain the original evidence artifact.
5. Control mapping
   Link evidence to the appropriate control and sovereignty dimension.
6. Evaluation
   Execute the relevant test and evaluate Pass, Partial, Fail, Exception or Unknown.
7. Dashboard and reporting
   Update the control posture, maturity assessment and audit trail.

Example: GPU sovereignty control

Suppose the control says:

```
Compute: AI workloads run on approved sovereign infrastructure.
```

Our process:

1. OpenShift reports which nodes host the AI workloads.
2. NVIDIA DCGM reports the GPU hardware and telemetry.
3. Dell iDRAC supplies the corresponding physical server inventory.
4. The inventory crosswalk maps physical GPU hardware to the approved infrastructure.
5. The policy engine verifies that the workload is on an approved host and within the approved site.
6. Evidence is generated and linked to the Compute control.

Only if all mandatory assessment criteria pass should the control receive Pass.

A healthy GPU alone does not prove data residency, approved workload placement or sovereign ownership.

10. The evidence record we should standardize

Every collector should send the same logical record format.

```
None
{
  "evidence_id": "EV-000001",
  "control_id": "CMP-C01",
  "dimension": "Compute",
  "environment": "ENV1",
  "source": "NVIDIA_DCGM",
  "collector_id": "COL-GPU-001",
  "collected_at": "2026-11-01T10:00:00Z",
  "status": "COLLECTED",
  "integrity": {
    "algorithm": "SHA-256",
    "hash": "<calculated-hash>",
    "signature_reference": "<signature-id>"
  },
  "payload_reference": "<internal-object-storage-reference>",
```

```
  "validation": "PENDING"
}
```

This is a proposed application contract, not a claim that the control baseline already contains these exact fields.

I would maintain two separate states:

- Evidence collection state: Collected, Failed, Missing, Expired.
- Control assessment state: Pass, Partial, Fail, Exception, Unknown.

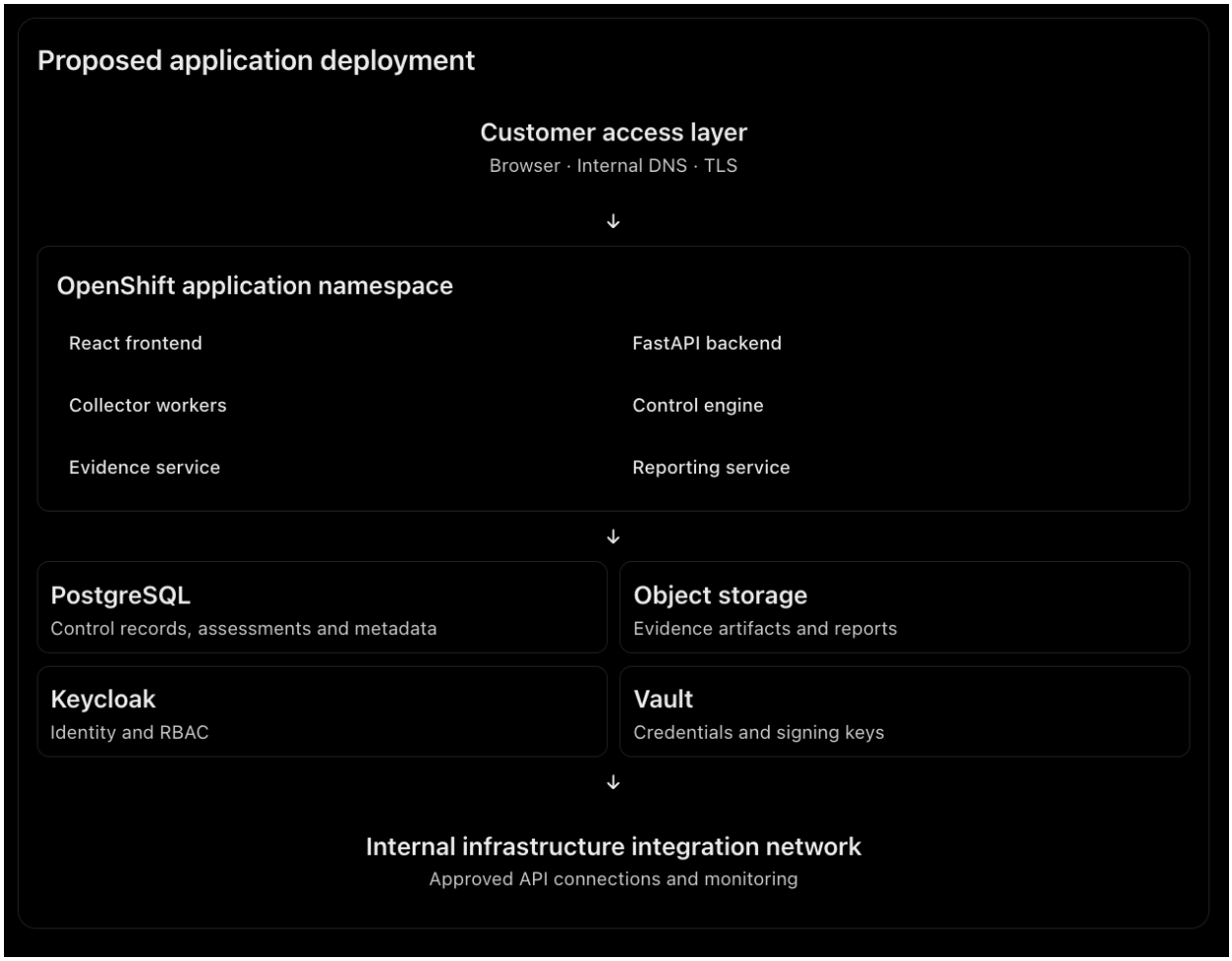
That separation is essential. A collector succeeding does not automatically mean the control passes.

PART 4 — Where should our application run?

11. Recommended deployment architecture

My recommendation is to deploy the Control Tower inside the customer's sovereign OpenShift environment.

This is important because the application should continue operating even if public internet connectivity is unavailable.



We should separate the application workloads from the AI workloads through dedicated namespaces, network policies, resource quotas and service accounts.

Suggested namespace structure

```
None
sovereign-control-tower
│
├── sct-frontend
├── sct-backend
├── sct-collectors
```

```
├── sct-assurance
├── sct-evidence
├── sct-database
├── sct-identity
├── sct-monitoring
│
├── ai-gateway
├── ai-model-serving
└── ai-rag-services
```

Some components, particularly monitoring and identity, may be shared infrastructure services rather than dedicated application deployments. The namespace structure is a logical proposal and should follow the cluster team's platform standards.

12. Technology stack I propose

Layer	Technology	Why we use it
Frontend	React + TypeScript	Enterprise dashboard
UI	Tailwind CSS + reusable components	EY-inspired design

Backend	Python + FastAPI	API and integration services
Database	PostgreSQL	Controls,users, assessments
ORM	SQLAlchemy	Database access
Migrations	Alembic	Schema versioning
Authentication	Keycloak	OIDC and identity
Secrets	HashiCorp Vault	Credential protection
Evidence storage	S3-compatible object storage	Evidence artifacts
Integrity	SHA-256 + digital signatures	Evidence verification
Infrastructure API	Dell Redfish	Server inventory
Virtualization	Nutanix Prism API	Cluster inventory
Container platform	OpenShift	Application deployment

GPU telemetry	NVIDIA DCGM Exporter	GPU monitoring
Metrics	Prometheus	Time-series metrics
AI inference	vLLM	Local LLM serving
Model orchestration	KServe	Model deployment
Container registry	Quay	Approved images
Security	ACS	Container and workload security
Deployment	Helm + GitOps	Repeatable deployments
Observability	Prometheus + Grafana	System monitoring

> For our two-week development milestone, we do not need to install and configure every component ourselves. We should reuse components already available in the team's infrastructure.

PART 5 — What I need from the infrastructure team

This is the section I would actually take to your team meeting.

13. Infrastructure integration readiness checklist

Technical prerequisites request

Proposed information and access required before integration development.

A. Dell hardware

- XC760xa and R760 inventory
- Number of physical servers
- iDRAC IP addresses / DNS
- iDRAC version and firmware
- Redfish API access
- Read-only credentials
- TLS certificates and management VLAN

B. NVIDIA

- Exact GPU model and quantity
- GPU-to-server mapping
- Driver and CUDA versions
- DCGM version
- GPU Operator status
- DCGM Exporter endpoint
- Prometheus endpoint and read-only access

C. Nutanix

- Prism Central URL and version
- API documentation
- API service account
- Cluster and host inventory
- AHV configuration
- Flow policy access

D. OpenShift

- Cluster API URL
- OpenShift version

- Service account and RBAC
- Internal image registry
- GPU worker node inventory
- Internal DNS and certificate authority

E. Networking and security

- VLAN and subnet diagram
- Firewall rules
- DNS and NTP
- Bastion access
- SIEM integration details
- Offline software mirror

F. Application services

- PostgreSQL deployment decision
- Object storage endpoint
- Keycloak realm/client
- Vault authentication method
- Quay and ACS API details
- Internal Git repository and GitOps

How the team should provide endpoints

I would ask the team to give us an endpoint inventory like this:

System	Endpoint example	Authentication
iDRAC	`https://idrac01.internal`	Read-only service account
Prism	`https://prism.internal:9440`	API service account

OpenShift	`https://api.cluster.internal:6443`	Service account
Prometheus	`https://prometheus.internal`	Read-only access
Vault	`https://vault.internal`	Approved auth method
Keycloak	`https://keycloak.internal`	OIDC client
Quay	`https://quay.internal`	Scoped token

These are examples of endpoint formats, not actual team endpoints.

Security requirement: Credentials should be provisioned through the approved secrets-management process, not sent over WhatsApp, email or placed in source code.

PART 6 — Two-week delivery roadmap

15. The plan I would propose to management

Target: working pilot in 10 working days

Day	Work	Deliverable
1	Architecture, endpoint inventory, development environment	Technical foundation

2	React, FastAPI, PostgreSQL and authentication skeleton	Working application
3	Eight dimensions, 44-control catalogue and database models	Control management
4	Evidence ingestion, hashing and storage	Evidence pipeline
5	Control evaluation engine and control detail page	Assurance workflow
6	Dell Redfish and Nutanix collectors	Infrastructure inventory
7	NVIDIA DCGM, Prometheus and OpenShift collectors	GPU and platform monitoring
8	Dashboard integration, exceptions and approvals	Functional operations UI
9	Maturity assessment, initial reports and AI model inventory	Governance features
10	Integration testing, offline deployment and demo	Pilot release

Dependencies: This assumes that the cluster, network and service endpoints are made available in parallel. If the infrastructure is not ready, we can complete mock integration and demonstrate the collector contracts, but cannot claim live hardware integration.

What we should deliver at the end

1. A functional Executive Overview.

2. All eight sovereignty dimensions.
3. Imported 44-control baseline, subject to validation against the original spreadsheet.
4. Environment-specific control status.
5. Working evidence ingestion and integrity verification.
6. A collector framework.
7. At least three live integrations if access is available.
8. Infrastructure and GPU monitoring.
9. Initial maturity assessment.
10. An assurance report.
11. An offline deployment package and runbook.

#imp asks

Decision	My recommendation
Infrastructure integration	API-driven collectors, not frontend-to-hardware connections
Hardware monitoring	Dell Redfish + NVIDIA DCGM + Prometheus
Application deployment	Inside the sovereign OpenShift environment
Evidence architecture	PostgreSQL metadata + protected object storage
Assurance architecture	Evidence-driven rules across all eight dimensions

1. Where will the Control Tower run?

Will it run on the same OpenShift cluster that hosts customer AI workloads, or on a separate management cluster?

My preference is a logically isolated management cluster or dedicated management node pool where capacity permits. It reduces the risk that a failure in an AI workload affects the governance platform.

2. Who owns the infrastructure collectors?

We need an agreed boundary between the infrastructure team and the application team. The infrastructure team provides endpoints, access and source-side configuration; we develop and maintain the collector framework and evidence processing.

3. What is the actual GPU deployment topology?

We must know whether NVIDIA GPUs are assigned directly to OpenShift bare-metal worker nodes, passed through to VMs, or managed through another supported virtualization configuration. This affects drivers, GPU Operator compatibility and telemetry collection. NVIDIA's compatibility requirements must be checked against the exact OpenShift and GPU Operator releases.